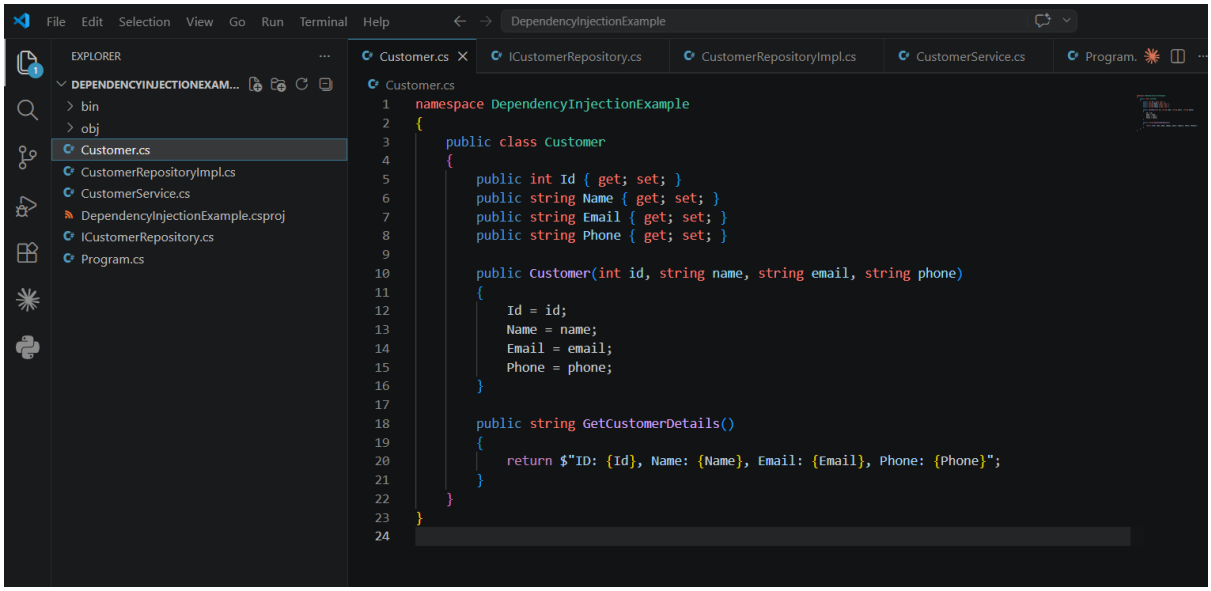
Hands-On Report

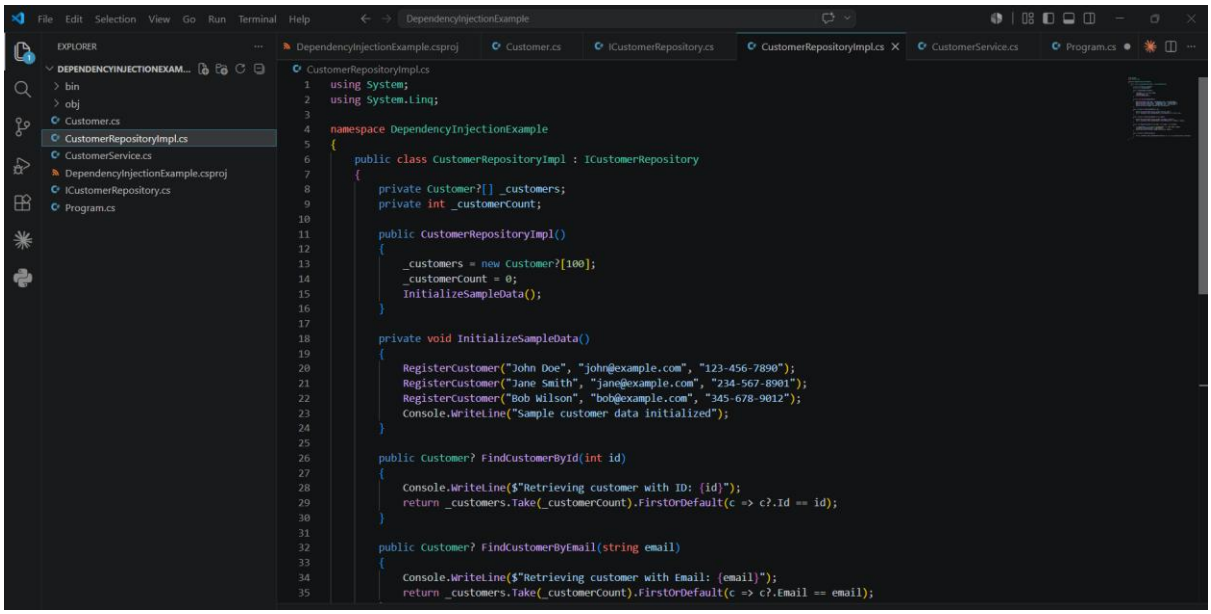
Exercise 11: Implementing Dependency Injection

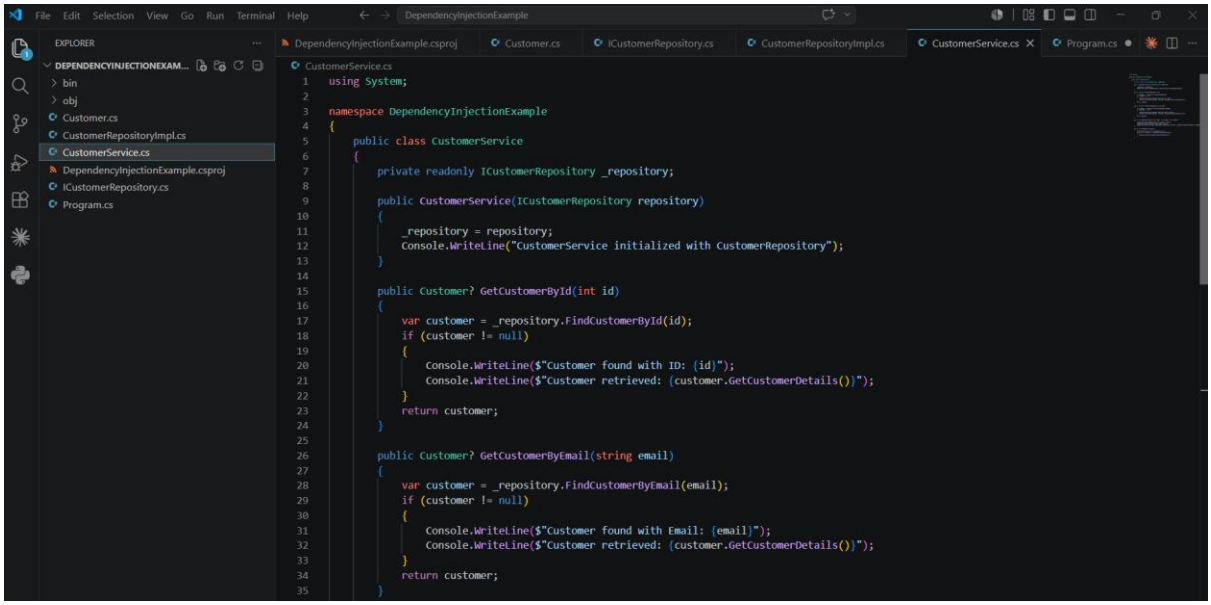
In this exercise, I implemented and tested **Dependency Injection** in a .NET application by creating a project named **DependencyInjectionExample**. I developed a **CustomerRepository** interface and a **CustomerRepositoryImpl** class to manage customer data. A **CustomerService** class was created to depend on the repository interface rather than a specific implementation, and constructor-based dependency injection was used to provide the required dependency.

After executing the application, I verified that the service successfully retrieved customer information through the injected repository. This practical exercise helped me understand how Dependency Injection reduces tight coupling between components, improves flexibility, enhances code maintainability, and makes .NET applications easier to test by following the Dependency Inversion Principle.

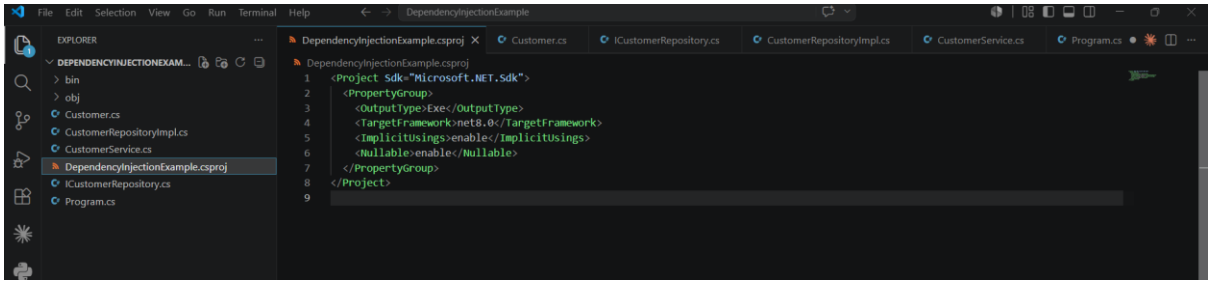
Coding:


File Edit Selection View Go Run Terminal Help
DependencyInjectionExample
EXPLORER
DEPENDENCYINJECTIONEXAM...
bin
obj
Customer.cs
CustomerRepositoryImpl.cs
CustomerService.cs
DependencyInjectionExample.csproj
ICustomerRepository.cs
Program.cs
Customer.cs
ICustomerRepository.cs
CustomerRepositoryImpl.cs
CustomerService.cs
Program.
Customer.cs
namespace DependencyInjectionExample
{
public class Customer
{
public int Id { get; set; }
public string Name { get; set; }
public string Email { get; set; }
public string Phone { get; set; }
public Customer(int id, string name, string email, string phone)
{
Id = id;
Name = name;
Email = email;
Phone = phone;
}
public string GetCustomerDetails()
{
return $"ID: {Id}, Name: {Name}, Email: {Email}, Phone: {Phone}";
}
}
}

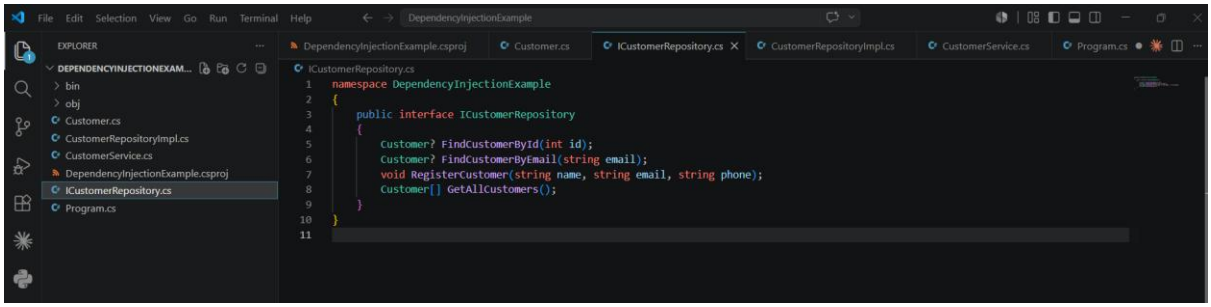

File Edit Selection View Go Run Terminal Help
DependencyInjectionExample
EXPLORER
DEPENDENCYINJECTIONEXAM...
bin
obj
Customer.cs
CustomerRepositoryImpl.cs
CustomerService.cs
DependencyInjectionExample.csproj
ICustomerRepository.cs
Program.cs
DependencyInjectionExample.csproj
Customer.cs
ICustomerRepository.cs
CustomerRepositoryImpl.cs
CustomerService.cs
Program.cs
CustomerRepositoryImpl.cs
using System;
using System.Linq;
namespace DependencyInjectionExample
{
public class CustomerRepositoryImpl : ICustomerRepository
{
private Customer?[] _customers;
private int _customerCount;
public CustomerRepositoryImpl()
{
_customers = new Customer?[100];
_customerCount = 0;
InitializeSampleData();
}
private void InitializeSampleData()
{
RegisterCustomer("John Doe", "john@example.com", "123-456-7890");
RegisterCustomer("Jane Smith", "jane@example.com", "234-567-8901");
RegisterCustomer("Bob Wilson", "bob@example.com", "345-678-9012");
Console.WriteLine("Sample customer data initialized");
}
public Customer? FindCustomerById(int id)
{
Console.WriteLine($"Retrieving customer with ID: {id}");
return _customers.Take(_customerCount).FirstOrDefault(c => c?.Id == id);
}
public Customer? FindCustomerByEmail(string email)
{
Console.WriteLine($"Retrieving customer with Email: {email}");
return _customers.Take(_customerCount).FirstOrDefault(c => c?.Email == email);



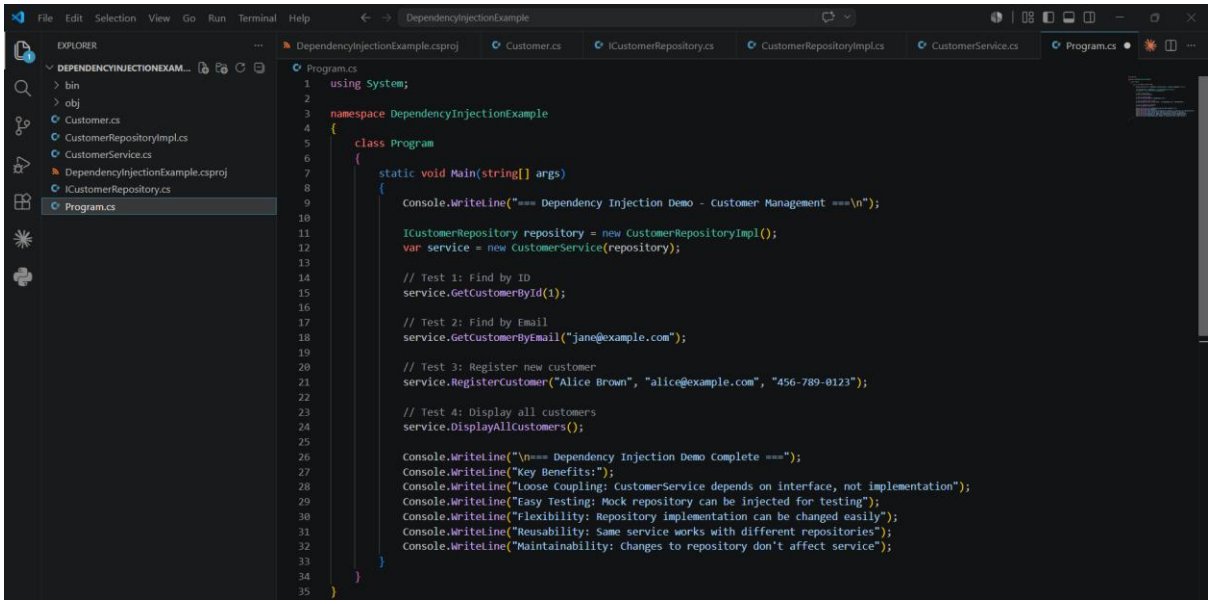
```
using System;

namespace DependencyInjectionExample
{
    public class CustomerService
    {
        private readonly ICustomerRepository _repository;

        public CustomerService(ICustomerRepository repository)
        {
            _repository = repository;
            Console.WriteLine("CustomerService initialized with CustomerRepository");
        }

        public Customer? GetCustomerById(int id)
        {
            var customer = _repository.FindCustomerById(id);
            if (customer != null)
            {
                Console.WriteLine($"Customer found with ID: {id}");
                Console.WriteLine($"Customer retrieved: {customer.GetCustomerDetails()}");
            }
            return customer;
        }

        public Customer? GetCustomerByEmail(string email)
        {
            var customer = _repository.FindCustomerByEmail(email);
            if (customer != null)
            {
                Console.WriteLine($"Customer found with Email: {email}");
                Console.WriteLine($"Customer retrieved: {customer.GetCustomerDetails()}");
            }
            return customer;
        }
```

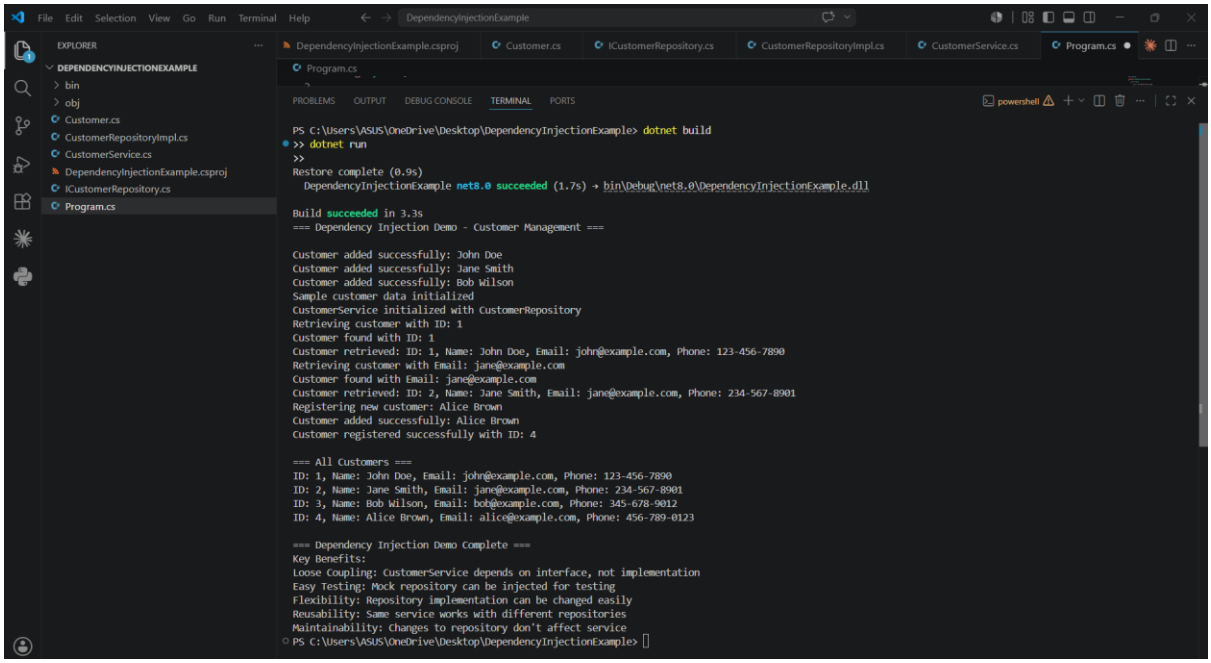


```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net8.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>
</Project>
```



```
namespace DependencyInjectionExample
{
    public interface ICustomerRepository
    {
        Customer? FindCustomerById(int id);
        Customer? FindCustomerByEmail(string email);
        void RegisterCustomer(string name, string email, string phone);
        Customer[] GetAllCustomers();
    }
}
```


File Edit Selection View Go Run Terminal Help
DependencyInjectionExample
EXPLORER
DEPENDENCYINJECTIONEXAM...
bin
obj
Customer.cs
CustomerRepositoryImpl.cs
CustomerService.cs
DependencyInjectionExample.csproj
ICustomerRepository.cs
Program.cs
DependencyInjectionExample.csproj
Customer.cs
ICustomerRepository.cs
CustomerRepositoryImpl.cs
CustomerService.cs
Program.cs
Program.cs
using System;
namespace DependencyInjectionExample
{
class Program
{
static void Main(string[] args)
{
Console.WriteLine("=== Dependency Injection Demo - Customer Management ===\n");
ICustomerRepository repository = new CustomerRepositoryImpl();
var service = new CustomerService(repository);
// Test 1: Find by ID
service.GetCustomerById(1);
// Test 2: Find by Email
service.GetCustomerByEmail("jane@example.com");
// Test 3: Register new customer
service.RegisterCustomer("Alice Brown", "alice@example.com", "456-789-0123");
// Test 4: Display all customers
service.DisplayAllCustomers();
Console.WriteLine("\n=== Dependency Injection Demo Complete ===");
Console.WriteLine("Key Benefits:");
Console.WriteLine("Loose Coupling: CustomerService depends on interface, not implementation");
Console.WriteLine("Easy Testing: Mock repository can be injected for testing");
Console.WriteLine("Flexibility: Repository implementation can be changed easily");
Console.WriteLine("Reusability: Same service works with different repositories");
Console.WriteLine("Maintainability: Changes to repository don't affect service");
}
}
}


File Edit Selection View Go Run Terminal Help
DependencyInjectionExample
EXPLORER
DEPENDENCYINJECTIONEXAMPLE
bin
obj
Customer.cs
CustomerRepositoryImpl.cs
CustomerService.cs
DependencyInjectionExample.csproj
ICustomerRepository.cs
Program.cs
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
powershell
PS C:\Users\ASUS\OneDrive\Desktop\DependencyInjectionExample> dotnet build
>> dotnet run
>>
Restore complete (0.9s)
DependencyInjectionExample net8.0 succeeded (1.7s) → bin\Debug\net8.0\DependencyInjectionExample.dll
Build succeeded in 3.3s
=== Dependency Injection Demo - Customer Management ===
Customer added successfully: John Doe
Customer added successfully: Jane Smith
Customer added successfully: Bob Wilson
Sample customer data initialized
CustomerService initialized with CustomerRepository
Retrieving customer with ID: 1
Customer found with ID: 1
Customer retrieved: ID: 1, Name: John Doe, Email: john@example.com, Phone: 123-456-7890
Retrieving customer with Email: jane@example.com
Customer found with Email: jane@example.com
Customer retrieved: ID: 2, Name: Jane Smith, Email: jane@example.com, Phone: 234-567-8901
Registering new customer: Alice Brown
Customer added successfully: Alice Brown
Customer registered successfully with ID: 4
=== All Customers ===
ID: 1, Name: John Doe, Email: john@example.com, Phone: 123-456-7890
ID: 2, Name: Jane Smith, Email: jane@example.com, Phone: 234-567-8901
ID: 3, Name: Bob Wilson, Email: bob@example.com, Phone: 345-678-9012
ID: 4, Name: Alice Brown, Email: alice@example.com, Phone: 456-789-0123
=== Dependency Injection Demo Complete ===
Key Benefits:
Loose Coupling: CustomerService depends on interface, not implementation
Easy Testing: Mock repository can be injected for testing
Flexibility: Repository implementation can be changed easily
Reusability: Same service works with different repositories
Maintainability: Changes to repository don't affect service
PS C:\Users\ASUS\OneDrive\Desktop\DependencyInjectionExample>